Calculating Radio Signal Arrival Times in Inhomogeneous Media

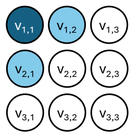
Marissa Boucher

Background

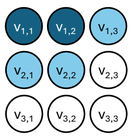
The Radio Neutrino Observatory in Greenland (RNO-G) seeks to detect ultrahigh energy (UHE) neutrinos via the Askaryan effect, which states that particles moving faster than the speed of light in a dense medium will produce radio signals. A large dielectric is needed to detect UHE neutrinos, so RNO-G has several antennas placed up to 100m into polar ice sheets [2]. One of the first steps in reconstructing a particle's properties from its radio emissions is to determine the signal arrival direction. This is done using **travel-time maps** to look up arrival times from possible interaction locations.

How do we make travel-time maps?

To generate travel-time maps for radio signals, we solve the wave equation in arbitrary inhomogeneous media. We make the approximation that refractive index varies only with depth, so travel-time maps are cylindrically symmetric, and the entire region can be described with radial (r) and depth (z) coordinates. We apply the Fast Marching Method (FMM) [3] using the Pykonal package in Python [4]. This method updates travel-times locally, allowing the solution to "spread" across the full domain.



Step 1: Set a starting point's travel-time and use a finite difference and the known velocity of each point (from index of refraction) to tentatively update its neighbors.



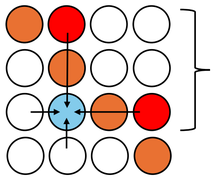
Step 2: Accept the point with the smallest travel-time. Tentatively update its neighbors. The Pykonal package may use a first- or second-order forward or backward difference depending on neighbor availability.

Step 3: Continue until there are no points with tentative travel-times. Not all points will have finite travel-times in cases if some points are unreachable (e.g., zero velocity at its neighbors).

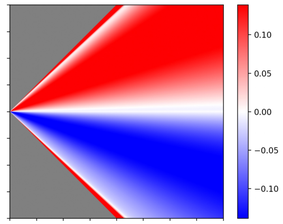
Problem

The Fast Marching Method in inhomogeneous media is subject to large numerical instability near boundary conditions. How do we achieve tolerable accuracy?

Applying the FMM in inhomogeneous media requires specialized boundary conditions, since the signal may change its direction due to refraction or reflection [1]. These conditions often follow a diagonal path, which leads to issues when the Pykonal package uses a second-order finite difference. To fix this problem, we forked Pykonal and adapted its FMM to exclusively use first-order differences, which meant that a finer grid was required to achieve the desired accuracy, but the numerical instabilities in the toy example on the right are no longer present.

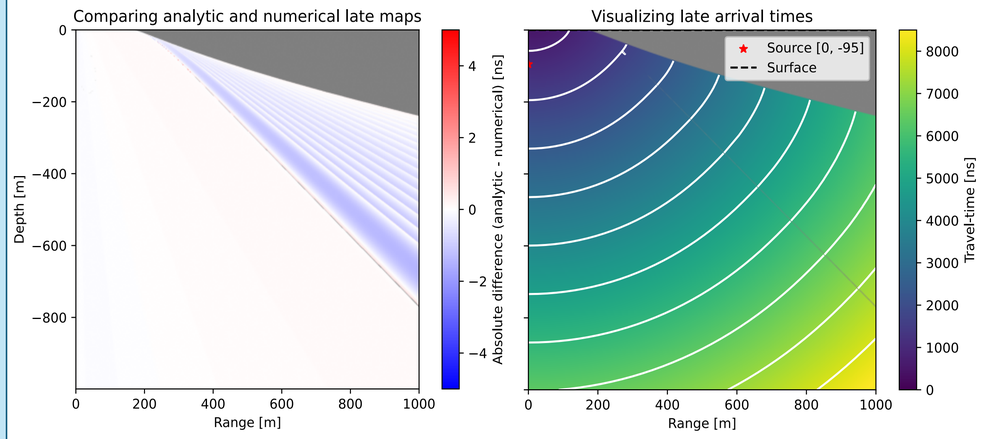


This side of the orange boundary condition is unreachable, but the 2 red circles are *included* in the finite difference used to update the desired traveltime.



A toy model in *homogeneous* media (velocity 1 m/s) where we compare Euclidean distance to the FMM solution. Note the large error "creeping in" along the boundaries. This is the result of second-order differences crossing boundary conditions.

Result: Travel-time map accuracy



Left: With the new fix, we find that in about 40 s of computation time we achieve <5 ns accuracy. Right: map of the absolute reflected & refracted travel-times – note the smooth contours!

Discussion & Future Work

Removing the solver's capabilities for second-order finite differences when updating neighboring travel-times does produce a general accuracy loss over the entire region. However, this loss can be easily mitigated by solving over a finer grid. The visualization shown in this poster has 16 million grid nodes and took about 40 seconds to generate, so time is not a limiting factor in this case. The boundary error from second-order differences, on the other hand, is not sensitive to grid size, so it is optimal to reduce to first-order solutions.

As of right now, my simulation only generates travel-times for the simplest approximation to the refractive index of Greenland ice, which is why there exists an analytic solution for comparison. In general, such a solution does not exist, making the need for a general numerical solver more urgent. I am currently generalizing the model further so it can properly supplement the existing RNO-G travel-time codebase.

Other extensions include adapting the current (Cartesian) coordinate system to account for the curvature of the Earth, which will allow calculations over multiple kilometers.

References

[1] Jean-David Benamou. In: *Journal of Computational Physics* (1996).
[2] C. Glaser et al. In: *The European Physical Journal* (2020).
[3] James A. Sethian. *Level set methods and fast marching methods.* 1999.
[4] Malcolm C. A. White et al. In: *Seismological Research Letters* (2020)

Packages used: NumPy, SciPy, Matplotlib, NuRadioMC [2], Pykonal [4]

Thanks to Nat Alden and Philipp Windischhofer for their ongoing support of this project.

For more information or to see the project, scan the QR code or visit github.com/mcb28/phys-250-final. Thank you for reading!